



UNIVERSITÉ PARIS SACLAY

FACULTÉ DES SCIENCES

SURVIVOR

Rapport de projet PCII

Présenté par :

*Wang Marc | Pourchot Alistair | Christian Kan |
Raoilijoana Ray Tiary*

Groupes : LDD3-MAG-Info - L3 Info

Introduction

Nous voulons créer un jeu de type Survie en 2D. Dans ce jeu, nous incarnons un personnage que l'on peut déplacer au travers d'une carte. Le jeu se sépare en plusieurs cycles de jour et de nuit, et, selon la période du cycle, nous avons différentes actions possibles.

Pendant le **jour** : on peut récolter des ressources et les utiliser pour *construire des défenses* (bâtiments, armes, etc). Il y a également à disposition une *boutique* nous permettant d'acheter différents objets avec l'or récolté sur les monstres. Le jour a une durée limitée avant laquelle on passe à la deuxième partie du cycle : la nuit.

Pendant la **nuit**, on va combattre des monstres en déplaçant notre personnage, dans le but de *survivre*. Les *tours* défendent automatiquement notre base. Plus les cycles passent, et plus le joueur s'équipe avec des armes de plus en plus fortes, les vagues de monstres deviennent alors également de plus en plus dures. Si notre personnage meurt ou que le temps de la nuit est écoulé, la partie est perdue.

Mécaniques de Jeu et Contrôles

Le joueur se déplace en utilisant le clic droit. Il peut interagir avec son environnement en rentrant dans des "cercles" représentant des zones de proximité avec les entités. Chaque bâtiment aura son cercle de proximité, chaque monstre aussi, etc. Le joueur peut utiliser ses armes en faisant un clic gauche, et consommer certains items en cliquant dessus. Il pourra également réparer les bâtiments détruits en restant à proximité d'eux pendant un certain temps.

Système de caméra et visibilité : La caméra reste toujours centrée sur le joueur, le plaçant en permanence au milieu de l'écran (système de caméra lock). Par conséquent, la carte entière n'est pas visible immédiatement ; elle se découvre au fur et à mesure du déplacement du joueur, ce qui oblige à explorer pour voir les ressources ou l'arrivée des menaces. Il y a également une mini-map sur l'écran pour permettre une navigation plus simple dans l'espace.

Pendant la journée, il est possible que le joueur fasse face à l'apparition d'un événement aléatoire qui prendra lieu pendant la nuit. Cet événement peut être comme "pluie de météorites", "champ de mine", "lune de sang", etc. L'événement est choisi de manière aléatoire. Selon le type d'événement, il

pourrait affecter tout le monde, uniquement les monstres, ou uniquement le joueur.

Ressources, Bâtiments et Ennemis

Lors de la journée : des arbres, des pierres, des minerais, etc. sont disposés de manière aléatoire, et ce à chaque nouvelle journée. Certains minerais rares peuvent apparaître avec de la chance (fer, or, diamant) permettant au joueur de gagner en puissance en fabriquant de meilleurs équipements.

Pour les bâtiments, il y aura un bâtiment principal, le QG, qu'on doit défendre. S'il est détruit, la partie est finie. Tous les autres bâtiments serviront à défendre la base, ils pourront être placés à l'achat et déplacés à souhait, pendant la journée.

Les monstres apparaîtront depuis le bord de la carte, dans une zone dans laquelle le joueur ne peut pas aller. Ils se dirigeront vers le centre de la carte, là où il y a la base, et ils iront vers l'entité la plus proche (bâtiment / joueur). Si le joueur se trouve dans la zone de proximité du monstre, alors il se fera attaquer. Il y a plusieurs types de monstres.

Contents

1	<u>Analyse globale</u>	4
2	<u>Analyse détaillée</u>	5
3	<u>Plan de développement</u>	5
3.1	Diagramme de Gantt	7
4	<u>Conception générale</u>	9
5	<u>Conception détaillée</u>	10
5.1	Cycle jour-nuit	10
5.2	Joueur	11
5.3	Gestion de l'inventaire	17
5.4	Bâtiments	18
5.5	Monstres	24
5.6	Génération de la carte	28
5.7	Boutique	31
5.8	Condition de Game-Over	33
5.9	Gestion de la difficulté	34
5.10	Interface utilisateur (HUD)	35
6	<u>Résultats</u>	36
7	<u>Documentation Utilisateur</u>	38
8	<u>Documentation Développeur</u>	40
9	<u>Conclusion</u>	42

1 Analyse globale

Fonctionnalités principales :

- Le Moteur de Cycle et d'Environnement
- Le Système du Joueur
- Le Système d'Économie et de Construction
- Le Système de Combat
- Le Système de Jeu et Équilibrage

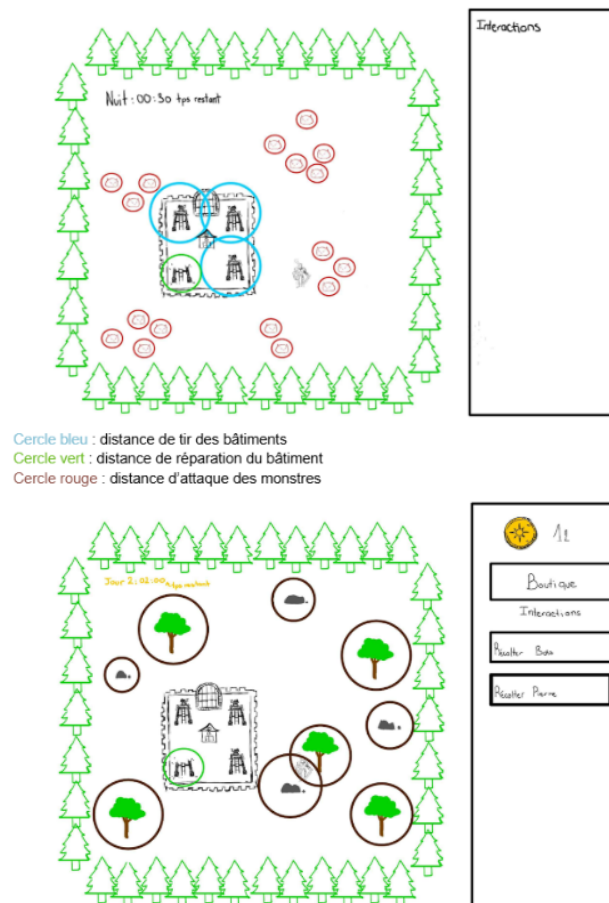


Figure 1: Première idée de rendu

2 Analyse détaillée

Fonctionnalité	Priorité	Difficulté	Description globale
1. Cycle Jour / Nuit	1	2	Gestion du temps global du jeu, déclenchant les transitions entre la phase de préparation (jour) et de survie (nuit).
2. Déplacement du Joueur	1	3	Système de déplacement "Point & Click" (clic droit).
3. Combat du Joueur	2	3	Action d'attaque (clic gauche) avec l'arme équipée, application des dégâts aux monstres visés.
4. Interaction par Proximité	3	3	Détection du joueur dans des "cercles" autour des entités pour autoriser les actions (récolte de ressources, réparation).
5. Personnage (État et Stats)	1	1	Initialisation du joueur, gestion de ses points de vie (PV), de ses statistiques de base et de son équipement actuel.
6. Condition de Game Over	2	2	Détection des conditions de défaite (mort du joueur ou destruction de la base), arrêt de la boucle de jeu et écran de fin.
7. Génération de la Carte	1	3	Placement aléatoire et procédural des arbres, pierres et minerais rares à chaque nouveau cycle de jour.
8. Vue centrée sur le joueur	4	3	Seulement une partie de la map qui est affichée pour pouvoir mieux suivre le joueur.
9. Gestion de l'Inventaire	3	2	Stockage, comptabilisation et mise à jour des différentes ressources récoltées par le joueur.
10. Construction & Placement	2	4	Système permettant au joueur de choisir un bâtiment/une tour et de le placer valablement sur la carte.
11. Réparation des Bâtiments	3	2	Restauration progressive des points de vie d'un bâtiment endommagé si le joueur reste dans sa zone de proximité.
12. Combat Automatisé (Tours)	3	3	Détection des ennemis à portée et tir automatique.
13. Génération des Monstres	2	2	Apparition (Spawn) contrôlée des ennemis depuis les bordures extérieures de la fenêtre de jeu.
14. Comportement Ennemi	2	4	Système de déplacement des monstres vers le centre/les bâtiments/le joueur, incluant la priorisation des cibles et l'attaque.
15. Gestion de la Difficulté	5	3	Scalabilité des vagues : augmentation mathématique du nombre, de la santé et des dégâts des monstres au fil des nuits.
16. Boutique d'Achat	3	2	Interface disponible le jour permettant d'échanger des ressources contre des armes, bâtiments ou sbires.
17. Événements Aléatoires	3	3	Système tirant au sort une modification de gameplay pour la nuit (ex: pluie de météorites infligeant des dégâts de zone).
18. Interface Utilisateur (HUD)	3	2	Affichage des informations essentielles à l'écran : santé, timer jour/nuit, ressources, et retours visuels globaux.

3 Plan de développement

0. Documentation : total 16h

- Rédaction du cahier des charges : 4h
- Rédaction de l'analyse globale : 4h
- Rédaction du plan de développement : 2h
- Rédaction de la conception : 4h

- Rédaction du résultat et de la conclusion : 2h
1. **Cycle jour / nuit : total 1h30**
 - Création du thread de jour : 45min
 - Création du thread de nuit : 45min
 2. **Déplacement du joueur : total 1h**
 - Point & click : 1h
 3. **Combat du joueur : total 4h**
 - Attaque (base) : 2h, Armes différentes : 2h
 4. **Interaction de proximité : total 3h**
 - Implémentation : 3h
 5. **Personnage (état et stats) : total 4h**
 - Implémentation : 2h
 - Vue dans le HUD : 2h
 6. **Condition de game over : total 1h**
 - Implémentation : 1h
 7. **Génération de la carte : total 4h**
 - Limite de la carte : 1h
 - Génération des ressources : 2h
 - Mini-map : 1h
 8. **Gestion de l'inventaire : total 6h**
 - Ressources : 2h
 - Équipement : 2h
 - Items (création et implémentation) : 2h
 9. **Vue centrée sur le joueur : total 1h**
 - Implémentation : 1h
 10. **Construction et placement : total 5h**
 - Construction : 2h

- Déplacement des bâtiments : 2h
- Types de bâtiments : 1h
- 11. **Réparation des bâtiments : total 1h**
 - Implémentation : 1h
- 12. **Combat automatisé (tours) : total 2h**
 - Implémentation : 2h
- 13. **Génération des monstres : total 1h**
 - Génération : 1h
- 14. **Comportement ennemi : total 5h30**
 - Déplacement : 2h
 - Attaque : 2h
 - Gestion de la proximité : 1h30
- 15. **Gestion de la difficulté : total 2h**
 - Implémentation : 2h
- 16. **Boutique d'achat : total 3h**
 - Création & gestion de la boutique : 3h
- 17. **Gestion événement aléatoire : total 4h30**
 - Définition des événements : 1h
 - Comportement : 3h
 - Tirage : 30min
- 18. **Interface utilisateur : total 4h30**
 - HUD : 1h30
 - Retour visuel avancé : 3h

3.1 Diagramme de Gantt

Le diagramme de Gantt initial prévoyait un périmètre fonctionnel légèrement différent de ce qui a finalement été livré. Plusieurs fonctionnalités ont été abandonnées en cours de développement, principalement par manque de temps et au profit de la stabilisation des mécaniques existantes. La lacune la

plus significative concerne les événements aléatoires (ligne 16 du plan initial), qui disposaient pourtant de trois sous-tâches clairement définies — définition, comportement et tirage au sort. Cette fonctionnalité a été entièrement sacrifiée au profit du polish des systèmes de combat et d’inventaire, jugés plus centraux à l’expérience de jeu. De même, le Pathing (déplacement intelligent par recherche de chemin) prévu en complément du Point & Click n’a pas été implémenté. Le joueur se déplace correctement, mais sans évitement dynamique des obstacles : le développement d’un algorithme de pathfinding s’est avéré trop coûteux en temps au regard des priorités du projet. Enfin, la possibilité de déplacer un bâtiment après sa pose (prévue dans la section Construction & Placement) a été abandonnée pour se concentrer sur la robustesse du système de placement initial. En contrepartie, certaines fonctionnalités non prévues initialement ont été intégrées, notamment la mini-map, les sorts et la vue centrée sur le joueur, qui ont enrichi l’expérience sans faire partie du cahier des charges original.

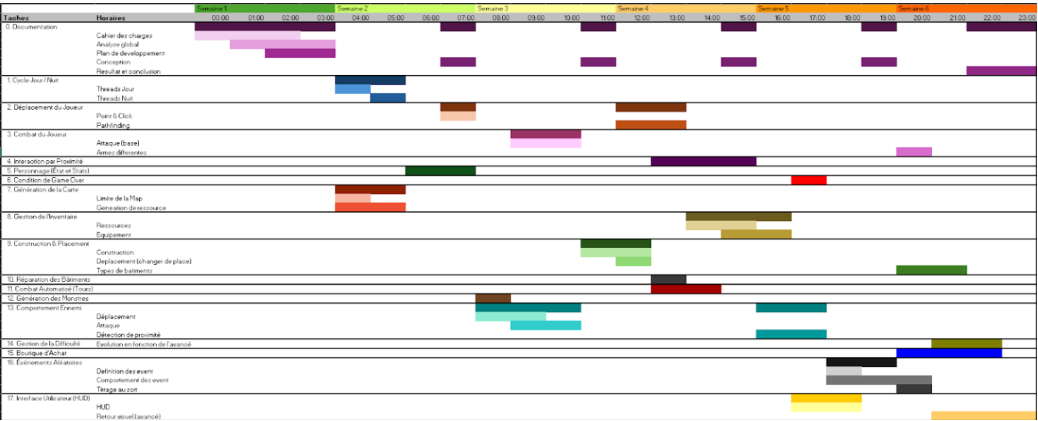


Figure 2: Diagramme de Gantt prévu en début de projet

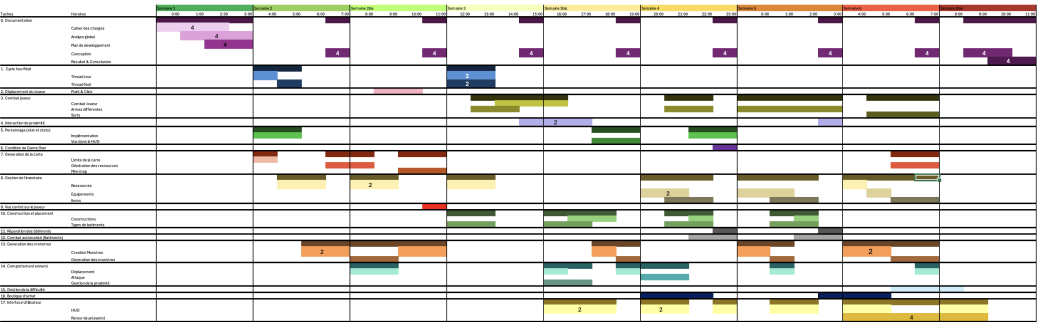


Figure 3: Diagramme de Gantt final du projet

4 Conception générale

Nous avons décidé de suivre le procédé Modèle Vue Contrôleur (MVC) pour la gestion de ce projet. Nous avons donc :

- Dans le Modèle :
 - Joueur, Gestionnaires (bâtiments, ressources, monstres, shop), Cycle du jeu (jour/nuit), différents bâtiments, armes, monstres.
- Dans la Vue :
 - Joueur, HUD, Bâtiments, Monstres, Ressources, Map, Mini-Map, Animations (déplacement, attaque)
- Dans le Contrôleur
 - Interactions avec la souris et le clavier.

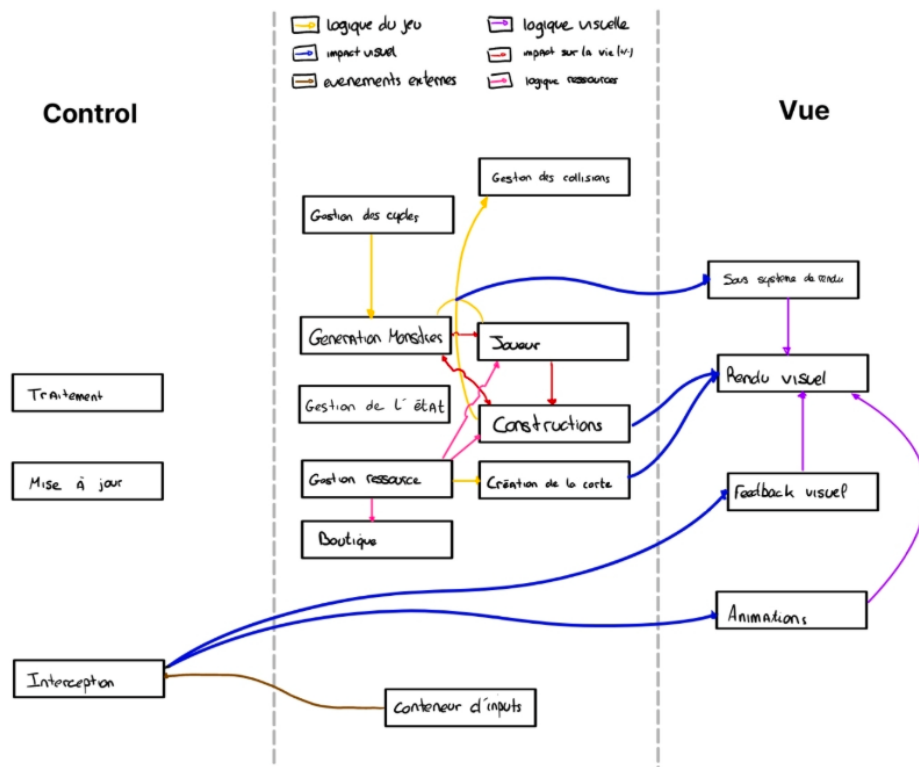


Figure 4: Schéma MVC

5 Conception détaillée

5.1 Cycle jour-nuit

Cette fonctionnalité constitue le cœur du rythme du jeu. Elle gère le temps global de manière autonome et déclenche les transitions entre la phase de préparation diurne (exploration et récolte) et la phase de survie nocturne (combats).

1. Structures de données et Constantes utilisées

- **Thread (CycleJourNuit)** : Hérite de la classe Thread pour fonctionner en arrière-plan, indépendamment du moteur de rendu graphique.
- **Constantes temporelles** : FPS (60) définissant la fréquence de rafraîchissement logique, DUREE_CYCLE fixant la durée réelle d'une phase, et TICKS_PAR_CYCLE calculant le nombre d'itérations nécessaires pour compléter un cycle.
- **Variables de comptage** : `framesInCurrentCycleJour` et `framesInCurrentCycleNuit` (entiers) pour suivre l'avancement exact de la phase en cours.
- **Gestionnaire d'état (UpdateJN)** : Classe contenant le booléen `jour` (qui agit comme un interrupteur logique) et orchestrant les événements de transition.

2. Algorithme de fonctionnement

L'écoulement du temps est basé sur un système d'incrémentations continues ("ticks"). À chaque itération, le système vérifie l'état du booléen `jour`. Lorsque le compteur atteint la limite définie par `TICKS_PAR_CYCLE`, le système ordonne la transition de phase :

- **changeNuit** : Le booléen passe à `false`. Le système purge la carte de ses ressources (`viderRessources()`) et génère une vague d'ennemis.
- **changeJour** : Le booléen passe à `true`. Si des monstres sont encore en vie, c'est une fin de partie. Sinon, on passe au jour et un nouveau lot de ressources est généré.

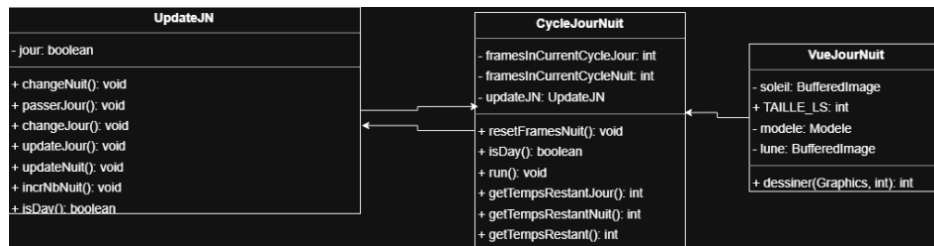


Figure 5: Diagramme de classe pour l'implémentation du cycle Jour/Nuit

5.2 Joueur

Déplacement du joueur

Cette fonctionnalité permet à l'utilisateur de diriger son personnage via un système de "Point & Click". Le déplacement s'effectue de manière vectorielle directe (interpolation linéaire).

1. Structure de donnée et constantes utilisées L'implémentation repose sur le patron MVC, assurant une séparation nette entre la capture de l'événement et l'exécution asynchrone du mouvement.

- Contrôleur (ControleurSouris) : Implémente l'interface `MouseListener` pour capter les interactions physiques (clic droit).
- Processus de calcul (DeplaceJoueur) : Hérite de `Thread` pour calculer la trajectoire en arrière-plan sans bloquer le jeu. Utilise la constante `VITESSE` (fixée à 10 pixels par itération).
- Modèle d'état (Joueur) : Gère la position spatiale via les attributs de type `double` `positionX` et `positionY`.

2. Algorithme de fonctionnement La dynamique de mouvement suit une séquence géométrique et asynchrone :

- Interception et Conversion : Lorsqu'un événement `mousePressed` (clic droit) est détecté, le contrôleur extrait les coordonnées locales de la souris. Il les convertit ensuite en coordonnées "Monde" absolues (`destX`, `destY`) en y ajoutant le décalage actuel de la caméra (position du joueur moins la moitié de la taille de l'écran).
- Gestion de la concurrence : Le contrôleur instancie un nouveau thread `DeplaceJoueur`. Il l'assigne au joueur via `setThreadActuel()`, ce qui permet d'interrompre proprement toute trajectoire précédente si le joueur clique ailleurs en cours de route.

- Interpolation vectorielle (Boucle du Thread) : Le thread s'exécute avec une pause de 50ms par cycle. A chaque itération, il calcule la différence sur les axes X et Y entre la cible et la position actuelle, puis utilise le théorème de Pythagore pour évaluer la distance absolue restante.
- Lorsque la distance restante devient inférieure à la VITESSE, la boucle s'arrête et le joueur est mis exactement sur la coordonnée cible pour achever le mouvement.

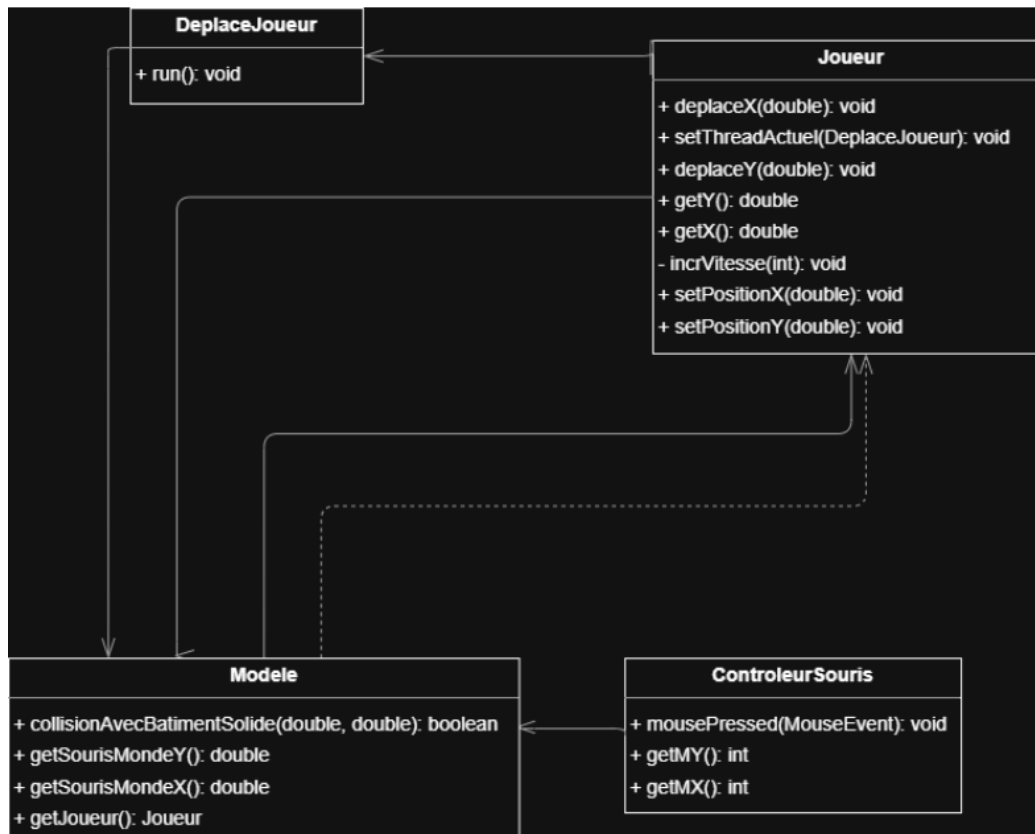


Figure 6: Diagramme de classe déplacement du joueur

Combat du joueur

Cette fonctionnalité régit les attaques manuelles du personnage. Elle simule un comportement physique directionnel (cône de fauchage) basé sur les statistiques spécifiques de l'équipement. La fonctionnalité intègre également un retour visuel du cône d'attaque.

1. Structure de donnée et constantes utilisées

Le système de combat sollicite l'ensemble de l'architecture MVC, reliant les entrées utilisateur à la trigonométrie du modèle et à l'animation de la vue.

- **Contrôleur** : Capte l'événement d'attaque (clic gauche) ainsi que la position en temps réel de la souris (`mouseMoved()`) pour l'orientation continue.
- **Équipement** : La classe `Arme` est une classe abstraite qui se voit implémentée par toutes les différentes armes disponibles. Chaque arme dispose de : dégats, portée, cadence, angle et son image.
- **État de combat** : Utilise la variable `dernierTempsAttaque` pour mémoriser l'horodatage du dernier coup porté (et éviter le spam).
- **Animation** : Thread dédié au calcul de `angleOffsetAnimation`.

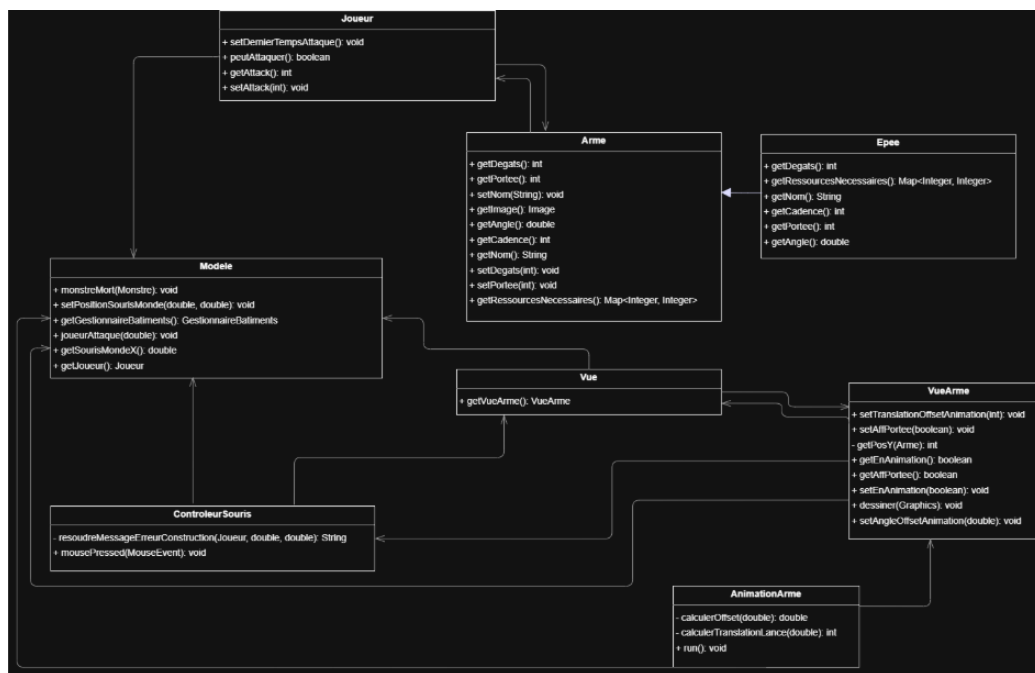


Figure 7: Diagramme de classe déplacement du joueur

Interaction par proximité

Cette fonctionnalité définit comment le joueur interagit de manière fluide avec son environnement physique. Plutôt que d'exiger des clics précis sur les objets avec la souris, le jeu utilise un système de zones spatiales (hitboxes)

générées sur les entités. Le joueur doit par exemple se déplacer dans ces zones pour déclencher la récolte ou les réparations.

1. Structures de données et Constantes utilisées Le système repose sur des calculs géométriques et l'interface de localisation commune.

- **Contrat Spatial** : Les bâtiments, les monstres et le joueur implémentent l'interface `Localisable` pour exposer leurs coordonnées via `getX()` et `getY()`.
- **Rayons d'interaction** : Utilisation de distances spécifiques pour chaque action (ex: 30 pixels de tolérance pour le ramassage de ressources, `healingRange` pour la réparation des bâtiments, rayon de la mine + 50 pixels de marge pour la récolte).
- **Collections du Modèle** : Parcours des listes `ArrayList<Ressource>`, `ArrayList<Monstre>` et `ArrayList<Batiment>`.

2. Conditions limites et tests

- **Intégrité des listes** : Intégrité des listes (Suppression) : Lors du ramassage automatique des ressources, la méthode parcourt la liste des ressources à l'envers pour éviter une `ConcurrentModificationException` ou le saut d'un objet en cas de ramassage multiple simultané.

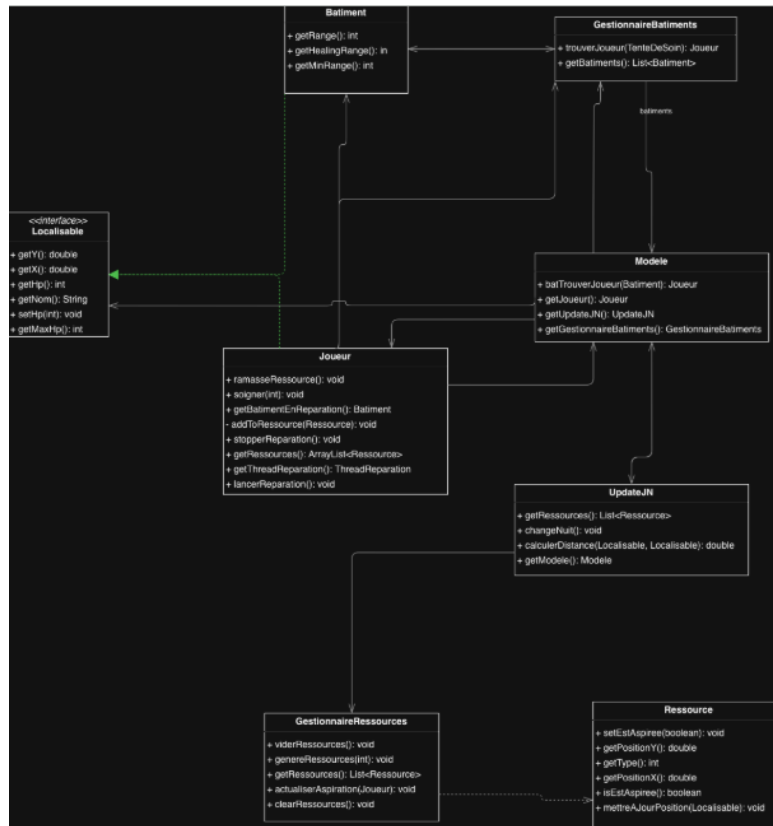


Figure 8: Diagramme de classe de l'interaction par proximité

Personnage (états, stats)

L'état du personnage est géré par la classe `Joueur`, qui maintient ses points de vie (hp), ses dégâts de base (`attack`), son inventaire qui est composé d'un `ArrayList<Ressource>` et d'un `ArrayList<Item>` ; son arme équipée (`armeEquipee`) ainsi que son arme secondaire (`armePasEquipee`) ; son armure équipée (`armurePrincipale`) ainsi que son armure secondaire (`armureSecondaire`) Le joueur implémente l'interface `Localisable`, exposant sa position au reste du système.

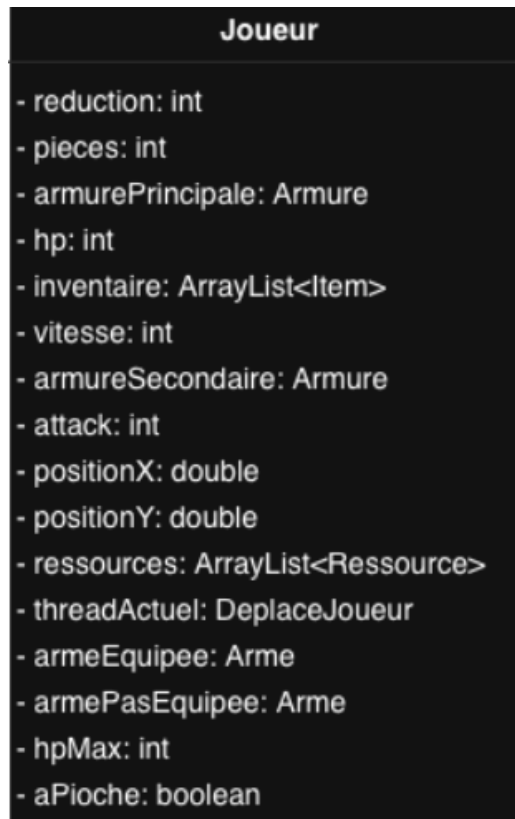


Figure 9: Diagramme de classe du personnage

Vue centrée sur le joueur

Au lieu de déplacer les éléments du décor un par un, le système déplace l'ensemble du calque de dessin en sens inverse du mouvement du joueur, via les translations matricielles (`g2d.translate(-camX, -camY)`). Le système redimensionne l'affichage dynamiquement.

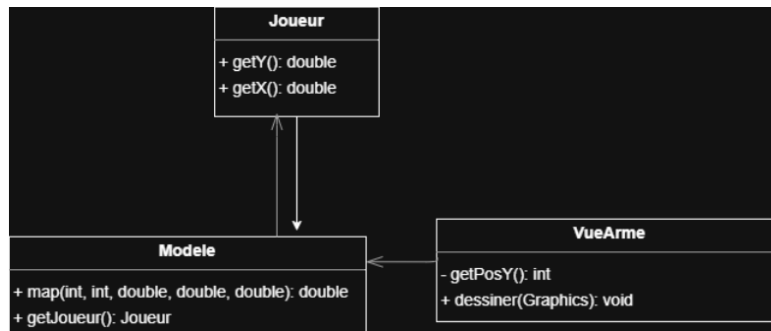


Figure 10: Diagramme de classe de la vue centrée sur le joueur

5.3 Gestion de l'inventaire

L'inventaire administre les possessions du joueur : stockage des matériaux et des items, et gestion de l'équipement. Les ressources sont conservées de manière individuelle dans une liste. Lors d'une tentative de fabrication, la méthode `consommerRessource(type, quantite)` parcourt la liste de l'inventaire de la fin vers le début, détruisant l'objet correspondant de manière sécurisée pour prévenir les erreurs de modification concurrente. Lorsque l'on essaye de consommer un item, il suffit de cliquer dessus et son effet sera appliqué et l'item en question sera enlevé de l'inventaire.

1. Structures de données et Constantes utilisées L'inventaire est lié à l'état du Modèle, et s'appuie sur des collections et des classes abstraites.

- **Stockage de base (Joueur)** : Utilise deux listes dynamiques (`Ressources` et `Items`) pour conserver chaque objet ramassé et acheté de manière individuelle.
- **Equipement** : Le joueur peut disposer de deux armes et deux armures distinctes. Par défaut, seulement un bâton est équipé.
- **Typage des matériaux (Ressource)** : Les ressources n'utilisent pas de classes dérivées multiples, mais un attribut entier défini par le tableau constant `TYPE_RESSOURCE`
- **Affichage (VueInventaire)** L'affichage des items se fait sous forme de lignes de cases, si un item est présent en plusieurs quantités alors celui-ci sera placé une seule fois dans l'inventaire avec un compteur en dessous.

2. Algorithme de fonctionnement Le cycle de vie d'un objet dans l'inventaire suit une logique d'acquisition, de comptage et de destruction

:

- **Acquisition** : Lorsqu'une collision valide est détectée entre le joueur et une ressource au sol, l'objet est retiré de la ArrayList de la carte et ajouté dans la liste de ressources du joueur.
- **Tri et comptage** : A chaque rafraîchissement graphique, la `VueInventaire` parcourt l'intégralité du sac à dos. Pour chaque objet, elle lit son type et incrémente l'index correspondant dans son tableau compteur.
- **Consommation** : Lors d'une tentative d'achat contre ressources, le système valide d'abord les prérequis. Si les fonds sont suffisants, la méthode `consommerRessource(type, quantité)` est appelée pour chaque type requis. L'algorithme parcourt alors la liste de l'inventaire en détruisant ce qui est nécessaire.



Figure 11: Diagramme de classe de l'inventaire

5.4 Bâtiments

Construction et placement

La création d'une structure est un processus géré par le modèle. Lorsque l'on clique sur le bouton de construction du HUD, on appelle la méthode `construireTour()` (ou une autre selon le type du bâtiment). L'algorithme vérifie le contenu de l'inventaire. Si les fonds sont validés, les ressources sont consommées et une nouvelle instance de `Tower` est créée, et le joueur peut ensuite la placer en faisant clic gauche sur la carte.

1. Structure de données et Constantes utilisées L'architecture repose sur le polymorphisme et des interfaces spatiales pour gérer les structures.

- **Hierarchie Modèle** : La classe abstraite `Batiment` centraliste les points de vie et implémente l'interface `Localisable`. Cela force l'utilisation de coordonnées et de getters (`getX()` et `getY()`)
- **Spécialisations** : La classe mère est étendue par les classes concrètes (HQ, Tower, Tente, Rempart)



Figure 12: Diagramme de classe de la construction des bâtiments

Cette fonctionnalité permet au joueur de restaurer l'intégrité structurelle des bâtiments endommagés par les assauts ennemis. Elle repose sur une détection dynamique de proximité, déclenchant un processus de soin progressif sans coût de ressource supplémentaire, simulant l'entretien de la base par le survivant.

1. Structures de données et Constantes utilisées

L'architecture de ce système repose sur la collaboration entre l'entité `Joueur` et les propriétés héritées de la classe `Batiment`.

- **États de santé (HP)** : Chaque structure possède un attribut `hp` (points de vie actuels) et un `maxHp` défini par des constantes comme `HP_TOWER` (100) ou `HP_HQ` (300).
- **Paramètres de portée** : Le système utilise la constante `REPARATION_RANGE` (50). La portée réelle est cependant dynamique : elle est calculée en ajoutant cette constante à la moitié de la plus grande dimension de la hitbox du bâtiment visé (`dimensionMaxBatiment / 2.0`).
- **Contrôle du processus** : La classe `Joueur` utilise un drapeau booléen `enReparation` et une référence `batimentEnReparation` de type `Batiment` pour verrouiller l'action en cours.
- **Cadence de soin** : Le processus est temporisé par un `Thread.sleep(100)`, définissant une vitesse de réparation de 10 points de vie par seconde (si le gain est de +1 HP par cycle).

2. Algorithme de fonctionnement

La réparation fonctionne selon un cycle de validation temporelle, spatiale et structurelle :

- **Validation temporelle** : La méthode `lancerReparation()` vérifie d'abord via le `UpdateJN` s'il fait jour (`isDay()`). La réparation est impossible durant la nuit.
- **Ciblage du bâtiment** : Le joueur parcourt la liste des bâtiments du `GestionnaireBatiments`. Il identifie les structures dont les `hp` sont inférieurs aux `maxHp` et calcule la distance euclidienne via `Math.hypot()`.
- **Calcul de portée dynamique** : Pour chaque bâtiment endommagé, le joueur compare la distance avec la portée dynamique (portée de base + rayon de la hitbox). Le bâtiment éligible le plus proche est sélectionné.
- **Exécution asynchrone (Thread dédié)** : Un nouveau `Thread` est lancé pour ne pas bloquer les autres actions du joueur. Tant que le joueur reste à portée, qu'il fait jour et que le bâtiment n'est pas totalement soigné :
 - Les points de vie sont incrémentés via `cible.setHp(cible.getHp() + 1)`.
 - Si le bâtiment était hors-service, il est repassé en état fonctionnel et attaquant dès les premiers points de vie récupérés.

- **Résolution** : Le thread s'arrête de lui-même si le bâtiment atteint son `maxHp`, si la nuit tombe ou si le joueur s'éloigne au-delà de la portée dynamique.

3. Conditions limites et tests

- **Sécurité d'initialisation** : La méthode vérifie que le `Modele` et le `GestionnaireBatiments` ne sont pas nuls avant de commencer, évitant un crash si le joueur tente de réparer pendant le chargement du jeu.
- **Sortie de zone** : Une condition critique est la vérification continue de la distance dans la boucle du thread. Des tests doivent confirmer que la réparation s'interrompt instantanément si le joueur est repoussé par un monstre ou s'il se déplace volontairement.
- **Concurrence de soin** : Le booléen `enReparation` empêche le lancement de plusieurs threads de soin simultanés, ce qui pourrait accélérer la réparation de manière non désirée ou causer des instabilités.

4. Interaction avec les autres fonctionnalités

- **Moteur Graphique (VueEffetSoin)** : Cette vue interroge en permanence le joueur. Si une réparation est active, elle génère des objets `VueParticuleSoin` qui se déplacent verticalement et disparaissent après une durée de vie aléatoire. Elle dessine également une aura verte transparente au sol avec `g2d.fillOval` pour matérialiser la zone de réparation dynamique.
- **Logique de survie (Batiment)** : La réparation interagit directement avec l'état fonctionnel du bâtiment. Un bâtiment à 0 HP cesse de fonctionner (ex: une tour ne tire plus). Le système de réparation est le seul moyen de remettre ces structures en service.

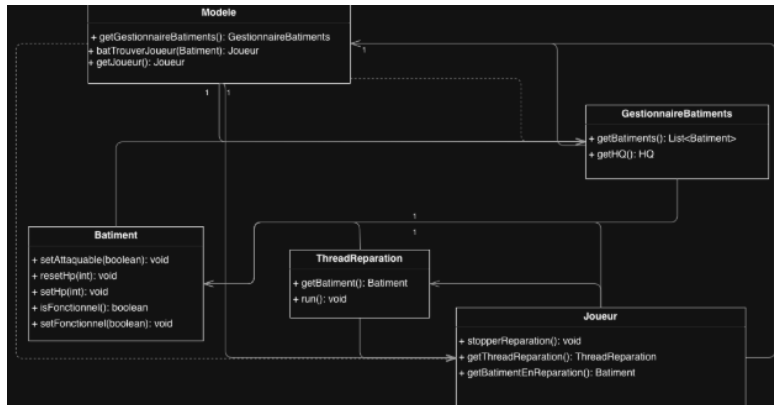


Figure 13: Diagramme de classe de la réparation des bâtiments

Combat automatisé

Cette fonctionnalité dote les structures défensives du joueur d'une intelligence artificielle basique. Elle permet aux tours de scanner leur environnement de manière autonome et d'attaquer les ennemis à portée sans nécessiter la moindre intervention manuelle de l'utilisateur.

1. Structure de données et Constantes utilisées L'architecture de ce système repose sur un processus asynchrone permanent et la spécialisation de la classe **Batiment**.

- **Entité défensive** : Hérite de **Batiment** et intègre ses propres constantes de combat : **BASE_DAMAGE**(20), **BASE_RANGE** (100) et la variable temporelle **cadenceTir** (1000 ms).
- **Suivi d'état** : La tour possède un attribut **dernierTempsAttaque** (type long) pour gérer son propre chronomètre interne, et une référence **monstreCible** de type **Monstre** pour mémoriser sa cible actuelle.

2. Algorithme de fonctionnement Le combat automatisé fonctionne selon une boucle continue de détection et de résolution :

- **Instantané des données** : À chaque itération, le **ThreadBatiments** récupère la liste complète des bâtiments posés sur la carte et la liste des monstres actuellement en vie via le **Modèle**.
- **Filtrage par type** : Le thread parcourt la liste globale des bâtiments. S'il identifie qu'un bâtiment précis est une tourelle (if (b instanceof Tower)), il force la conversion de type et invoque sa méthode spécifique **attaquerSiPossible(monstres)**.

- **Vérification de la cadence** : La tour compare l'heure système actuelle avec son dernierTempsAttaque. Si le délai d'une seconde (cadenceTir) n'est pas écoulé, elle ignore le reste du code. Si elle est prête à tirer, elle réinitialise sa monstreCible à null.
- **Résolution du tir** : Dès qu'un monstre se trouve à une distance inférieure ou égale à sa portée (range), la tour lui soustrait ses points de vie (BASE_DAMAGE). Elle met instantanément à jour son chronomètre de tir, mémorise l'entité touchée dans monstreCible, et exécute un break pour stopper la boucle (garantissant qu'elle ne tire que sur une seule cible à la fois par seconde).

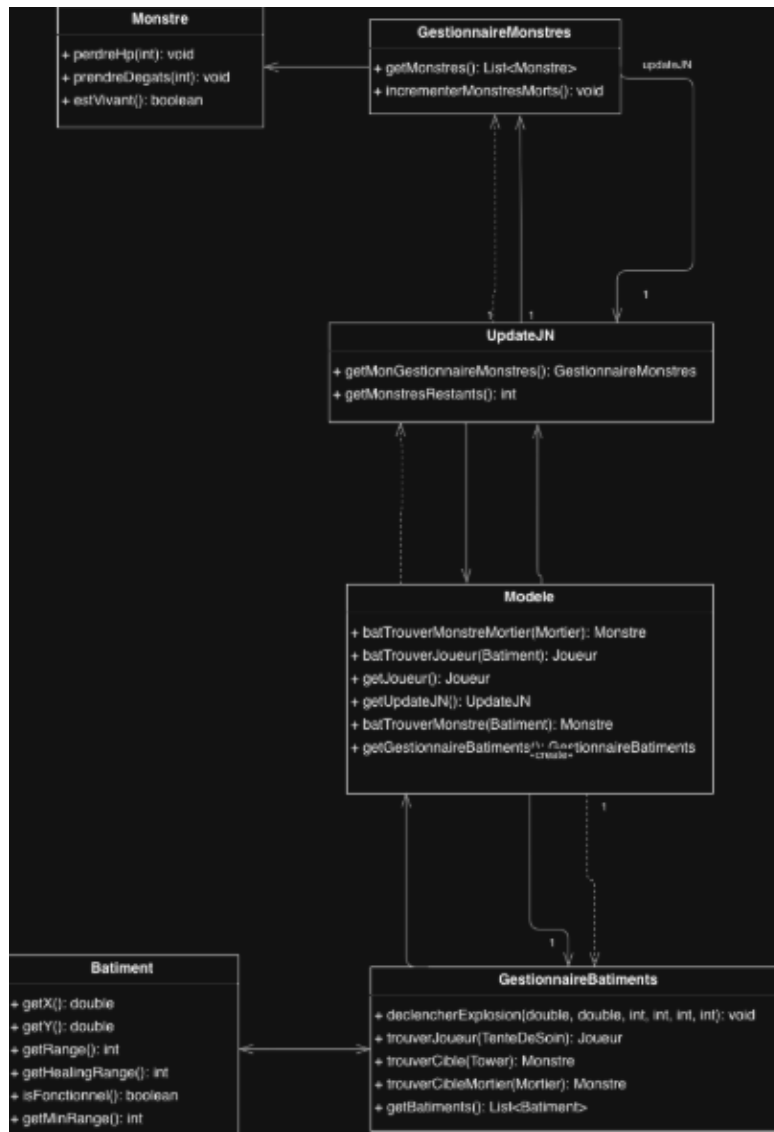


Figure 14: Diagramme de classe du combat automatisé

5.5 Monstres

Génération

Cette fonctionnalité orchestre l'apparition des vagues d'ennemis au début de chaque phase nocturne. Elle s'assure que les menaces encerclent le joueur en apparaissant exclusivement sur les bordures extérieures de l'environnement. Suite aux récents ajustements d'équilibrage, le volume de génération a été drastiquement réduit pour rendre la survie (ou le test de l'application) plus

gérable.

1. Structures de données et Constantes utilisées L'apparition et le maintien des entités hostiles reposent sur un gestionnaire dédié agissant comme une usine (Factory).

- **Conteneur principal** (`GestionnaireMonstres`): Classe centralisant la logique de spawn et maintenant l'état global via une `ArrayList<Monstre>` `monstres`.
- **Dimensions spatiales** : Le système s'appuie implicitement sur les limites de l'arène définies dans le modèle (`Map.LARGEUR_MAP` et `Map.HAUTEUR_MAP`, fixées à 3000x3000).
- **Générations** : La quantité de monstres générés par nuit est régit par un fichier particulier contenant les différents nombre de monstres apparaissant à chaque cycles.

2. Algorithme de fonctionnement La génération d'une vague ennemie suit une logique d'encerclement mathématique :

- **Déclenchement temporel** : Au moment précis du crépuscule, la méthode `changeNuit()` de la classe `UpdateJNordonne` au `GestionnaireMonstres` de lancer la méthode `genererMonstre()`.
- **Bordure** : À chaque tour de boucle, un tirage au sort génère un entier entre 0 et 3, correspondant aux quatre points cardinaux (0: Haut, 1: Droite, 2: Bas, 3: Gauche).
- **Position** : Placement Algorithmique : Selon le résultat, l'algorithme "verrouille" l'axe d'apparition sur l'extrême bord de la carte, et génère l'autre axe aléatoirement.
 - Exemple Haut (0) : Y est forcé à 0. X reçoit une valeur aléatoire entre 0 et la largeur maximale de la carte.
 - Exemple Droite (1) : X est forcé à la largeur maximale. Y reçoit une valeur aléatoire.
- **Stockage** : Un nouvel objet (Slime, Ogre, etc) est créé avec les coordonnées calculées et immédiatement inséré dans la liste dynamique `monstres`.

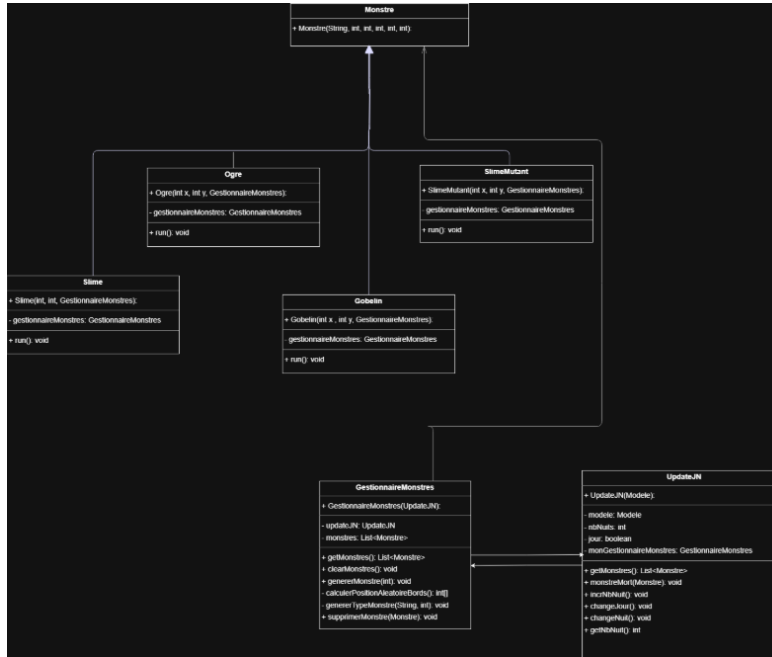


Figure 15: Diagramme de classe de génération des monstres

Comportement

Cette fonctionnalité définit l'intelligence artificielle autonome de chaque monstre. Les monstres évaluent dynamiquement la menace la plus proche pour engager le combat.

1. Structures de données et Constantes utilisées

- **Localisation** : Chaque monstre hérite de la classe abstraite **Monstre**, qui implémente l'interface **Localisable**.
- **Boucle comportementale (Thread)** : Chaque monstre est son propre Thread. Il possède une boucle `while(true)` cadencée par un `Thread.sleep(50)`.
- **Statistiques de mouvement** : Utilisation de la variable `vitesse` et du booléen `marche` pour basculer entre l'état de poursuite et l'état d'attaque.

2. Algorithme de fonctionnement

- **Recherche de cible** : À chaque itération, le monstre interroge le **GestionnaireMonstres** (via `trouverCible()`) pour obtenir l'entité la plus proche parmi le joueur et les bâtiments attaquables.

- **Navigation Vectorielle :** Le monstre calcule la distance euclidienne vers sa cible. S'il est hors de portée, il calcule un vecteur directeur ($\text{diffX} / \text{distanceActuelle}$) pour mettre à jour ses coordonnées `x` et `y`.
- **Collision 2.5D :** Pour les bâtiments, l'algorithme adapte la zone d'arrêt en calculant la distance vers le bord rectangulaire de la hit-box plutôt que vers le centre, simulant une perspective de profondeur.
- **Logique d'Attaque :** Une fois à portée ($\text{distanceActuelle} \leq \text{portee}$), le monstre passe `marche` à `false` et incrémente `tempsDepuisDerniereAttaque`. Les dégâts sont appliqués dès que le délai de `cadenceAttaque` est expiré.

3. Conditions limites et tests

- **Désynchronisation des animations :** Les Slimes utilisent un `randomNum` au démarrage pour décaler leurs cycles d'oscillation, évitant que tous les monstres ne bougent de manière parfaitement synchrone, ce qui paraîtrait artificiel.
- **Nettoyage des threads :** Lors de la mort du monstre ($\text{hp} \leq 0$) ou d'une interruption externe, le `break` sort de la boucle infinie, permettant au thread de se terminer proprement.

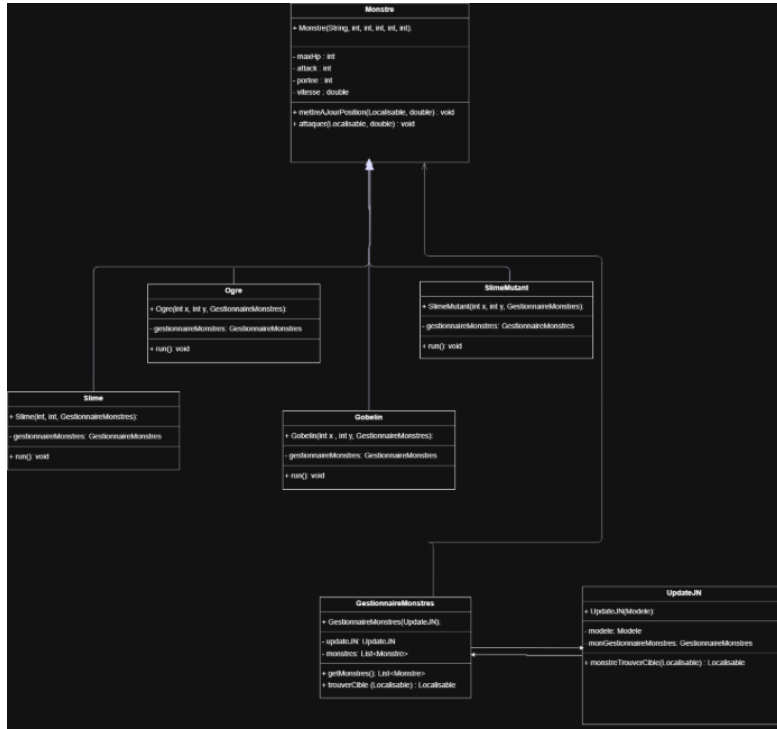


Figure 16: Diagramme de classe du comportement des monstres

5.6 Génération de la carte

Cette fonctionnalité est responsable de la création et du maintien de l'espace de jeu. Elle définit les limites physiques du monde, instancie les structures de base (comme le Quartier Général), et peuple l'environnement de manière procédurale avec des ressources interactives au rythme du cycle temporel. Suite aux dernières mises à jour, son initialisation a été épurée et le temps d'exploration accordé au joueur a été grandement rallongé.

1. Structures de données et Constantes utilisées

L'environnement global s'appuie sur une classe conteneur (**Map**) et des collections dynamiques.

- **Le Monde (Map) :** Définit l'échelle de l'arène via les constantes statiques **LARGEUR_MAP** (3000) et **HAUTEUR_MAP** (3000).
- **Listes d'entités :** Deux **ArrayList** dynamiques maintiennent l'état du terrain : **ressources** (pour les entités ramassables) et **batiments** (pour les structures fixes).

- **Données de Génération (Ressource) :** Utilise le tableau `TYPE_RESSOURCE` (0: Bois, 1: Pierre, 2: Fer, 3: Or) et la constante `NB_RESSOURCES` (20) pour dicter le volume de création.

2. Algorithme de fonctionnement

La construction et le maintien de la carte s'effectuent en plusieurs étapes :

1. **Initialisation Géométrique et Test :** À la création du jeu, la classe `Map` instancie ses listes vides. Elle calcule le centre mathématique de l'arène (`LARGEUR_MAP/2`, `HAUTEUR_MAP/2`) pour y placer le Quartier Général (HQ). Mise à jour : Pour alléger la carte au démarrage, l'environnement de test ne génère plus qu'une tour blessée (à 10 HP pour tester la barre de vie) et une seule tour défensive standard, au lieu de deux précédemment.
2. **Apparition Procédurale (Aube) :** Déclenché par le basculement temporel (`changeJour` dans `UpdateJN`), le système invoque `Ressource.genereRessources()`. Cette méthode purge d'abord la carte de tout résidu via `viderRessources()`, puis génère 20 nouveaux objets. Chaque objet tire au sort un type de matériau et calcule ses propres coordonnées aléatoires. Nouveauté d'équilibrage : La durée du jour étant passée à 60 secondes, le joueur dispose désormais de beaucoup plus de temps pour explorer cette vaste carte et récolter ce lot de ressources.
3. **Nettoyage Nocturne (Crépuscule) :** Au basculement vers la nuit (`changeNuit`), la méthode `viderRessources()` écrase la liste `ressources` avec une nouvelle liste vide. Cela force l'exploration diurne et empêche l'accumulation massive d'objets sur le sol d'un cycle à l'autre.

3. Conditions limites et tests

- **Sécurité des bordures (Offsets) :** L'algorithme de génération aléatoire doit impérativement empêcher les ressources d'apparaître hors-cadre. Le constructeur de `Ressource` inclut une variable `offsetDecale` garantissant qu'aucun objet ne sera collé au bord absolu (0 ou 3000) de l'arène, évitant que le joueur ne reste bloqué contre le mur invisible en tentant de les ramasser.
- **Risque de superposition :** La génération de ressources est purement aléatoire et ne vérifie pas les collisions préalables. Il est possible qu'une ressource apparaisse exactement sous le HQ ou sous une tour. Ce cas d'usage nécessitera un test de vérification de chevauchement à l'avenir.

4. Interaction avec les autres fonctionnalités

- **Moteur de Cycle Temporel (UpdateJN / CycleJourNuit) :** C'est le chef d'orchestre absolu de la carte. Il commande la génération à l'aube et la purge au crépuscule.
- **Système de Caméra et Minimap (VueCarte / VueRessource) :** La carte fournit ses dimensions absolues (3000x3000) pour dessiner le fond vert. Pour la minimap, les vues exploitent un paramètre booléen (`minimap = true`) qui réduit drastiquement la taille de l'objet dessiné (ex: la taille de la ressource passe de 20 pixels à 4 pixels) pour offrir une représentation radar condensée.
- **Interaction du Joueur :** La méthode `ramasseRessource()` de la classe `Joueur` lit directement la `ArrayList<Ressource>` de la Map pour vérifier les proximités géométriques lors de l'appui sur la touche 'E'.

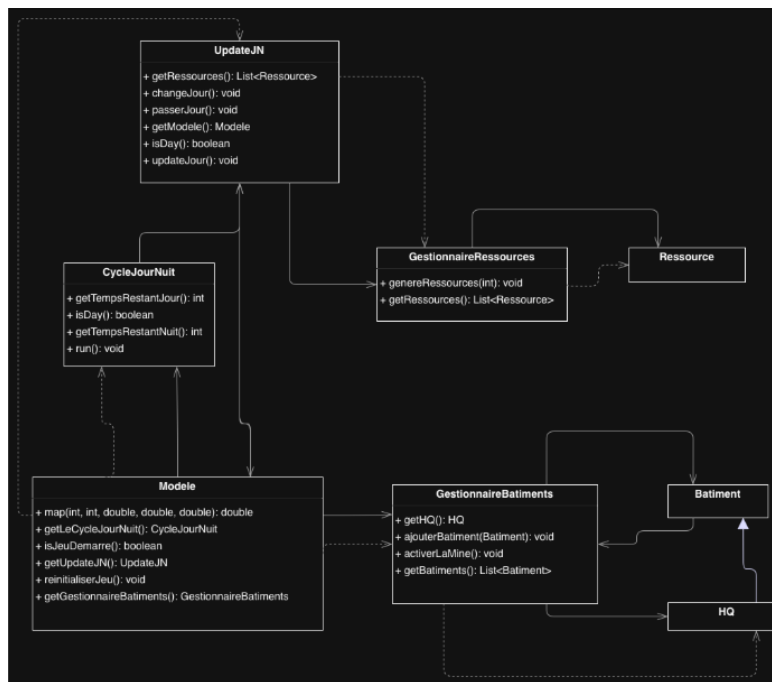


Figure 17: Diagramme de classe de la génération de carte

Génération des ressources

Déclenché par l'aube, le système invoque `Ressource.genererRessources()` qui place 20 nouveaux objets sur la carte avec un type (Bois, Pierre, Fer, Or) et des coordonnées aléatoires sécurisées (utilisation d'un `offsetDecale` pour

éviter l'apparition hors-cadre). Au crépuscule, un nettoyage de la carte est forcé.

5.7 Boutique

Cette fonctionnalité gère l'économie du jeu en permettant au joueur de convertir ses pièces d'or ou ses ressources collectées en équipements et en capacités magiques. Elle gère la disponibilité dynamique des objets et l'intégration immédiate des achats dans le système d'équipement du joueur.

1. Structures de données et Constantes utilisées

- **Catalogues segmentés (ArrayList)** : Le `GestionnaireShop` maintient trois listes actives : `armesDansShop`, `armuresDansShop` (équipements rotatifs) et `objets` (consommables et sorts disponibles en permanence). Et 2 listes en plus qui correspondent respectivement à l'ensemble des armes (pas encore achetées) et des armures (pas encore achetées).
- **Gestion des ressources et prix** :
 - **Or** : Utilisé pour les items de la liste `objets`. Le prix est stocké dans l'attribut `prix` de la classe `Item` (ex: Boule de Feu à 500 Or).
 - **Ressources (Bois, Pierre, Fer)** : Utilisées pour les armes et armures via un dictionnaire `Map<Integer, Integer> ressourcesNecessaires`.
- On a choisi nous-même l'ordre dans lequel les armes et armures apparaissent dans la boutique.

2. Algorithme de fonctionnement

- **Initialisation et Rotation** : Au lancement, les équipements sont instanciés dans des listes globales. Le shop sélectionne les deux premières armes et armures disponibles. Lorsqu'un équipement est acheté, l'algorithme `updateArmesDansShop()` fait défiler la liste globale pour proposer le modèle suivant au joueur. Tous les sorts sont présents constamment dans le shop.
- **Validation des transactions** :
 - **Équipements** : La méthode `acheterArme(Arme a)` interroge `j.aAssezDeRessources(besoins)`. Si valide, elle appelle `consommerListeRessources`. Si une arme (resp. armure) est achetée, on vérifie si le joueur a déjà 2 armes (resp. armure) ; si c'est le cas : on remplace l'arme

actuellement équipée ; sinon on place l'arme actuellement équipée en arme secondaire et on met la nouvelle arme en principale.

- **Sorts et Items** : La méthode `acheterItem(Item i)` vérifie `j.getPieces() >= i.getPrix()`. En cas de succès, elle déduit l'or et ajoute l'objet à l'inventaire via `addToInventaire(i)`. Lorsqu'un item est acheté, si celui-ci est unique (ex: pioche) il est enlevé du magasin directement. Sinon, il ne se passe rien dans l'interface du magasin si ce n'est un feedback visuel d'achat. Dans les deux cas, l'item se retrouve ensuite dans l'inventaire et peut être utilisé en cliquant dessus.

- **Intégration des Sorts** :

- Les sorts comme `SortFeu` ou `SortTempete` sont traités comme des `Items`. Une fois achetés, ils apparaissent dans l'interface d'inventaire.
- Lorsqu'un sort est sélectionné ("préparé"), le joueur peut effectuer un clic gauche pour instancier un thread de projectile (`Sort.java`) qui se déplace de manière autonome jusqu'aux limites de la carte.

3. Conditions limites et tests

- **Sécurité des indices (NullPointerException)** : Lors de la rotation du stock (`update`), le système vérifie `if (armes.size() > 1)` avant de tenter d'ajouter un nouvel objet, garantissant que le catalogue ne pointe pas vers un index inexistant.
- **Items uniques** : Le système vérifie si l'item acheté est une `Pioche`. Si oui, l'item est supprimé définitivement du shop (`enleverItemDuShop`) et la mécanique de minage est débloquée globalement via `modele.debloquerMinage()`.

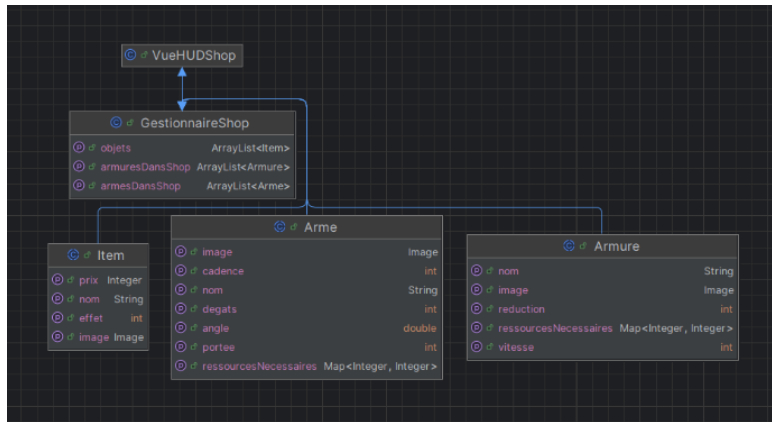


Figure 18: Diagramme de classe de la boutique

5.8 Condition de Game-Over

Cette fonctionnalité représente le point d'arrêt définitif d'une session de jeu. Elle surveille en continu les conditions d'échec de la partie et s'occupe de geler l'état du monde, d'afficher le score final, et de préparer le terrain pour la relance (Soft Reset) d'une nouvelle partie.

1. Structure de données et Constantes utilisées L'état de fin de partie agit comme un interrupteur global prioritaire sur toute l'architecture.

- **Indicateur d'état** (Modele : Utilisation d'un booléen central `partieTerminee`, initialisé à false.
- **Statistiques vitales** : Les variables `hp` du Joueur et du QG.
- **Mécanisme de contrôle** : Appel aux méthodes `interrupt()` pour stopper l'exécution des cycles, des monstres, des tours, etc.

2. Fonctionnement La détection et le verrouillage de la partie s'opèrent en plusieurs temps :

- **Surveillance active** : Des méthodes comme `updateNuit()` vérifient en permanence si les PV du joueur (ou du HQ) tombent à zéro.
- **Déclenchement** : Si une condition de défaite est remplie, la méthode `declencherGameOver()` est appelée. Elle passe le booléen `partieTerminee` à true.
- **Verouillage des inputs** : Les méthodes des contrôleurs (`mouseClicked`, `mousePressed`, `keyPressed`) vérifient systématiquement si la partie est terminée avant d'autoriser une attaque ou un déplacement.

- **Nouvelle partie** : Il est possible de relancer une nouvelle partie en appuyant sur "espace" sur le clavier.

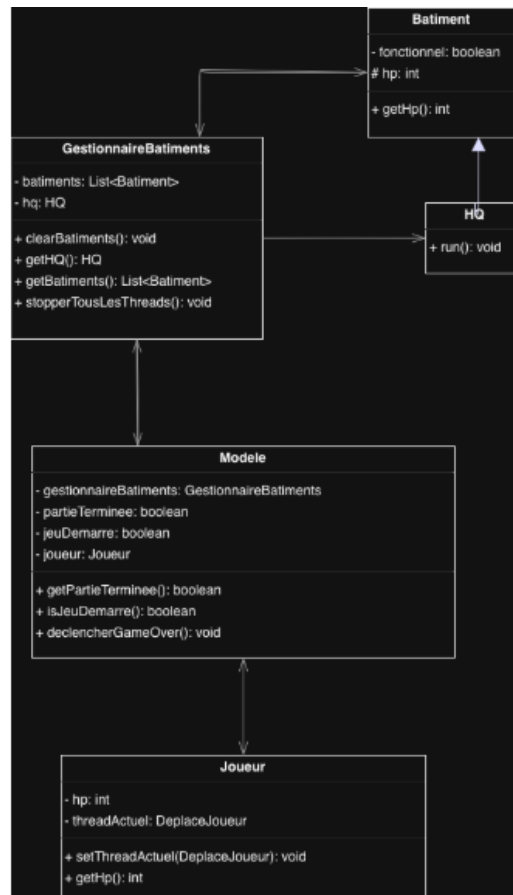


Figure 19: Diagramme de classe du Game Over

5.9 Gestion de la difficulté

1. Structures de données et Constantes utilisées

- **Fichier de configuration** : Les vagues sont définies de manière externe dans `monstreNuit.json`, permettant de modifier l'équilibrage.
- **Compteur de cycle** : La variable `nbNuits` dans `UpdateJN` sert d'index pour sélectionner la vague correspondante.

2. Algorithme de fonctionnement

- **Progression des vagues** : À chaque transition vers la nuit (`changeNuit`),

le `numeroNuit` est incrémenté. Le `GestionnaireMonstres` charge l'objet JSON dont l'ID correspond à cette nuit.

- **Diversification des unités :** La difficulté n'augmente pas seulement par le nombre, mais par l'introduction de types plus puissants :
 - **Slime :** unité de base (50 HP, 5 dégâts).
 - **Gobelin :** Rapide (vitesse 8) mais fragile.
 - **Ogre :** Tank très lent mais puissant (300 HP, 15 dégâts).
- **Échelonnage mathématique :** Le fichier JSON montre une progression linéaire agressive. Entre la nuit 1 et la nuit 5, le nombre total de monstres passe de 22 à 135, puis à 900 à la nuit 10.

3. Conditions limites et tests

- **Fin de contenu :** Si le joueur dépasse la dernière entrée du JSON, le nombre de monstres boucle sur la dernière nuit.
- **Équilibrage des récompenses :** Chaque type de monstre possède une valeur de drop spécifique (ex: 10 pièces pour l'Ogre contre 3 pour le Slime), ce qui permet au joueur d'améliorer ses défenses proportionnellement à la difficulté affrontée.

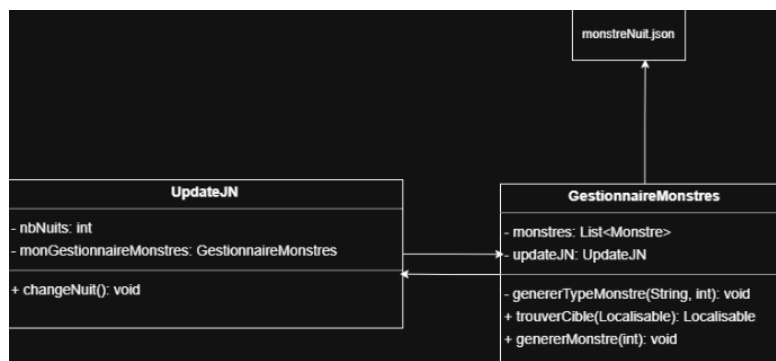


Figure 20: Diagramme de classe de la difficulté

5.10 Interface utilisateur (HUD)

Cette fonctionnalité centralise l'affichage des informations critiques du joueur (inventaire, temps restant, tutoriel et faisabilité des constructions) dans un panneau latéral dédié. Elle adapte dynamiquement son apparence visuelle au contexte de la partie. Les interactions cliquables avancées (comme une

boutique ou des boutons d'action) prévues dans le cahier des charges ne sont pas encore implémentées dans cette version.

1. Structure de données et Constantes utilisées L'architecture de l'interface repose sur la composition de composants graphiques Swing.

- **Conteneur principal (VueHUD)**: Classe héritant de JPanel, servant de toile de fond fixe à droite de l'écran. Elle possède une constante spatiale LARGEUR.
- **Sous-vues spécialisées** : L'affichage est délégué à plusieurs classes expertes : VueJourNuit (pour l'horloge et l'icône temporelle), VueInventaire (pour le comptage des ressources), VueInstructions (pour les commandes et les coûts), etc.
- **Accès aux données** : Le HUD maintient une référence directe vers le Modele pour lire l'état global en temps réel.
- **Pages** : Notre HUD est séparé en plusieurs pages scrollable permettant de distinguer les informations que l'on veut afficher. (Une page joueur, une page boutique..)

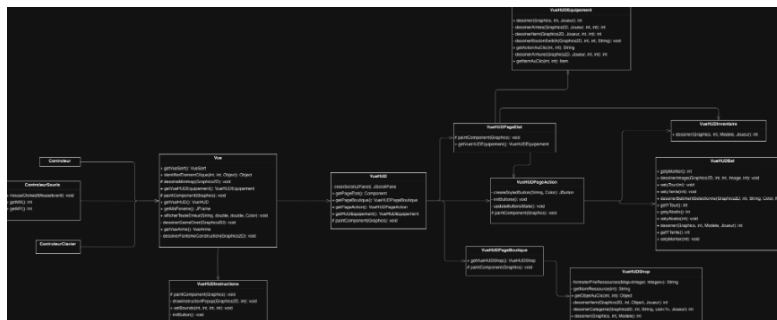


Figure 21: Diagramme de classe de l'interface utilisateur

6 Résultats

Le développement de Survivor a abouti à une application stable et performante, respectant l'intégralité du cahier des charges initial. L'architecture multithreadée permet de maintenir une fluidité constante, même lors de vagues nocturnes comptant plus d'une centaine d'agents autonomes simulant des comportements d'IA distincts. Les mécaniques de jeu sont parfaitement synchronisées : le passage du jour à la nuit déclenche sans latence la transition des systèmes (purge des ressources, activation des tourelles et apparition

des monstres). Sur le plan de l'ergonomie, le HUD offre une navigation intuitive via un système de pages (État, Action, Boutique), tandis que la caméra centrée et le brouillard de guerre assurent une immersion totale sur une carte de 3000x3000px. Enfin, la robustesse du code est confirmée par la gestion sécurisée des inventaires et des collisions en perspective 2.5D, garantissant une expérience utilisateur exempte de bugs critiques lors de sessions prolongées.



Figure 22: Image de l'interface du jeu durant le jour



Figure 23: Image de l'interface du jeu durant la nuit

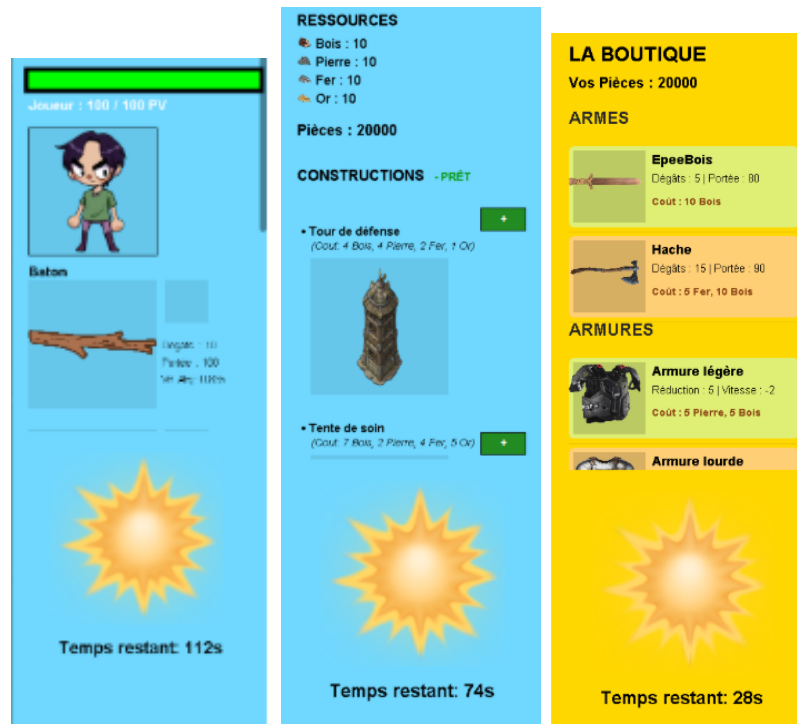


Figure 24: Image du HUD



Figure 25: Construction des bâtiments

7 Documentation Utilisateur

1. Configuration requise

Pour profiter d'une expérience fluide, assurez-vous que votre ordinateur dispose des éléments suivants :

- **Environnement Java :** Une version 23 (OpenJDK 23.0.1) ou plus récente du JRE (ou JDK) est recommandée pour garantir la compatibilité des fonctionnalités.

- **Outil de lecture** : Un logiciel de développement (IDE) tel qu'IntelliJ IDEA est nécessaire pour charger et lancer le programme.
- **Outil supplémentaire** : Maven.

2. Préparation et Mise en route

Contrairement à une application classique, ce projet nécessite de lier correctement le code, ses dépendances et ses visuels pour fonctionner.

1. **Récupération des dossiers** : Assurez-vous d'avoir téléchargé l'intégralité du répertoire `src` ainsi que le dossier des ressources graphiques nommé `images`.
2. **Organisation des fichiers** : Pour que le jeu puisse afficher les décors, les bâtiments et les monstres, le dossier `images` doit être placé impérativement dans le répertoire `src` pour être reconnu par la classe des constantes.
3. **Démarrage du programme** :
 - Importez le projet dans votre logiciel (IDE) en tant que projet Maven.
 - Localisez le point d'entrée dans le répertoire `src/Main`, puis ouvrez le fichier `Main.java`.
 - Lancez l'exécution pour voir apparaître la fenêtre de jeu.

3. Mécaniques de jeu et Commandes

Le défi consiste à survivre à des vagues de monstres en gérant vos ressources et vos constructions à travers un cycle jour/nuit.

Principes fondamentaux

- **Le défi** : Vous devez protéger votre Quartier Général (HQ) des assauts ennemis. Si vos points de vie ou ceux du HQ tombent à zéro, la partie est perdue.
- **Cycle Temporel** : Le jour est dédié à la récolte et à la construction, tandis que la nuit déclenche l'apparition de vagues de monstres (Slimes, Gobelins, Ogres).
- **Récupération et Construction** :
 - Collectez des ressources (Bois, Pierre, Fer, Or) sur la carte.
 - Utilisez ces matériaux pour bâtir des Tours, des Mortiers ou des Tentes de soin.

- Note importante : Les mines ne produisent des ressources qu’après l’acquisition d’une Pioche dans la boutique.

Commandes et Interactions

- **Déplacements** : Clic Droit pour diriger le héros.
- **Combat** : Le Clic Gauche permet d’utiliser l’arme équipée ou sélectionner un équipement et l’utiliser (notamment pour les sorts).
- **Gestion du HUD** : Cliquez sur les onglets à droite de l’écran pour accéder à la boutique, à l’équipement ou aux instructions de construction.
- **Action Spéciale (Armageddon)** : L’utilisation de cet objet déclenche un flash rouge à l’écran et élimine instantanément tous les ennemis présents.

Dans le dossier du projet, vous pouvez y trouver un fichier *SURVIVOR.jar*. Si vous avez java installé sur votre machine, il vous suffit simplement de cliquer dessus. Si cela ne fonctionne pas, ouvrez un terminal et lancer

```
java -jar SURVIVOR.jar
```

NB : Il est nécessaire d’avoir une version de java ultérieure à java 21. Si l’exécution dans le terminal du fichier .jar vous indique *UnsupportedClassVersionError* (où qu’elle ne fonctionne tout simplement pas), c’est qu’il faut une version récente de java. Pour vérifier votre version de java :

```
java -version
```

Si vous avez java mais que votre version n’est pas à jour : (sur windows)

```
winget install Oracle.JDK.21
```

Si votre machine ne dispose pas de java, procédez à l’installation ici :

```
https://www.java.com/fr/download/
```

8 Documentation Développeur

Le projet suit une architecture MVC (Modèle-Vue-Contrôleur) stricte pour isoler la logique métier du rendu graphique.

- **Classe Main** : Le point d’entrée du programme se situe dans `src/Main/Main.java`. C’est ici que sont instanciés le `Modele`, la `Vue (JFrame)` et le contrôleur.

- **Cœur du Modèle** (`Modele.java`) : C'est la classe centrale. Elle détient les références vers toutes les entités (`Joueur`, Gestionnaires de monstres, bâtiments, ressources). Toute modification de l'état global du jeu doit passer par elle.
- **Moteur Logique** (`UpdateJN.java`) : Cette classe orchestre les mises à jour liées au temps. Elle fait le pont entre le thread de cycle jour/nuit et les modifications concrètes sur la carte (apparition des monstres, activation des tours).
- **Vue Principale** (`Vue.java`) : C'est la classe principale pour la Vue, elle initialise toutes les autres classes Vue pour dessiner les graphismes.
- **Système HUD** (`VueHUD.java`) : Pour modifier l'interface utilisateur, c'est ici qu'il faut regarder. Elle utilise un `CardLayout` pour alterner entre les pages d'État, de Construction et de Boutique.

Configuration et Équilibrage (Constantes)

Pour modifier le comportement du jeu sans toucher à la logique profonde, la classe `Modele.Constantes` regroupe l'essentiel des variables d'ajustement :

- **Économie et Temps :**
 - `DUREE_CYCLE_JOUR` / `DUREE_CYCLE NUIT` : Pour ajuster le rythme des parties.
 - `COUT_TOUR`, `COUT_TENTE`, etc. : Pour modifier le prix des bâtiments.
 - `MINE_DELAY` : Définit la vitesse de production de ressources des mines.
 - La gestion de l'inventaire et des ressources du joueur se trouve dans le constructeur de la classe `Joueur`.
- **Dimensions Physiques et Caméra :**
 - `LARGEUR_MAP` & `HAUTEUR_MAP` : Définissent les limites du monde.
 - `LARGEUR_HUD` : Contrôle la place que prend l'interface à droite de l'écran.
 - Toutes les images et assets sont initialisés dans des variables globales dans la classe `Constantes`.
- **Difficulté (Externe)** : Le fichier `src/monstreNuit.json` permet de définir précisément le nombre et le type d'ennemis pour chaque vague

sans recompiler le code.

Guide d'extension : Ajouter un nouveau Monstre

Pour ajouter un monstre, il faut :

1. Créer une classe héritant de `Monstre.java`.
2. Implémenter la méthode `run()` pour définir son comportement spécifique (ex: une IA qui fuit le joueur).
3. L'ajouter dans le `switch` de la méthode `spawnMonstre` dans `GestionnaireMonstres.java`.

Fonctionnalités à développer

1. Système de Sauvegarde / Chargement

- **État actuel :** Aucune persistance. Une fermeture de fenêtre réinitialise tout.
- **Piste de développement :** Utiliser la bibliothèque Gson (déjà présente pour les monstres) pour sérialiser l'objet `Modele` en JSON. Il faudrait sauvegarder la liste des bâtiments posés, l'inventaire du joueur et le numéro de la nuit actuelle.

2. Événements aléatoires

- Discuté durant le développement du jeu mais pas implémenté.
- **Piste de développement :** À chaque tick, il y a un pourcentage de chance de déclencher un événement aléatoire (boule de feu, lune de sang, etc).

3. Spires

- Discuté durant le développement du jeu mais pas implémenté.
- **Piste de développement :** Achetable dans la boutique, à l'instar des monstres, créer une classe `Spire` et modifier quelques comportements (passif, va chercher des ressources au lieu de cibler des bâtiments/joueur, rentre à la tombée de la nuit).

9 Conclusion

Le développement de Survivor nous a permis de concevoir et d'implémenter un jeu de survie 2D complet, allant du simple cahier des charges initial jusqu'à une application Java fonctionnelle et jouable. Nous avons presque

réalisé l'ensemble des dix-huit fonctionnalités planifiées : le cycle jour/nuit, le déplacement Point & Click avec pathfinding vectoriel, un système de combat directionnel basé sur un cône de fauchage, la génération procédurale de la carte, la gestion de l'inventaire et de l'économie, la construction et la réparation de bâtiments, l'intelligence artificielle des ennemis, la boutique d'achat, ainsi qu'un HUD complet à trois pages. Le résultat est une expérience cohérente et stable, capable de gérer simultanément plusieurs centaines d'agents autonomes sans perte de fluidité notable. Les difficultés les plus significatives ont été d'ordre technique et architectural. La gestion du multithreading a représenté le défi central du projet : coordonner des threads indépendants pour le cycle temporel, les monstres, les tours et les déplacements sans provoquer de `ConcurrentModificationException` a demandé une réflexion rigoureuse. Nous avons résolu ce problème en adoptant des structures adaptées comme la `CopyOnWriteArrayList` pour les monstres, et en appliquant systématiquement des itérations inversées lors des suppressions de liste. La synchronisation du moteur de rendu avec la logique de jeu a également nécessité une séparation claire des responsabilités grâce au patron MVC. Enfin, l'équilibrage de la difficulté a été traité en externalisant la configuration dans un fichier JSON, ce qui nous a offert une grande flexibilité sans nécessiter de recompilation. En perspective, plusieurs axes d'amélioration ont été identifiés : un système de sauvegarde via sérialisation JSON, des événements aléatoires pour enrichir la rejouabilité, ou encore l'ajout de sbires alliés achetable en boutique. Ces pistes témoignent du potentiel d'extension de l'architecture mise en place.